

Využitie veľkých jazykových modelov v programovaní

Ročníkový projekt

Vladyslav Bilichenko

Obsah

1	Úvod	3
2	Analýza problému a teoretické východiská	4
2.1	Veľké jazykové modely a Prompt Engineering	4
2.2	Obmedzenia kontextového okna	4
2.3	Retrieval-Augmented Generation (RAG)	5
2.4	Hodnotenie kvality vygenerovaného kódu	5
2.5	Optimalizácia promptu: Diskrétny problém batoha	5
3	Návrh riešenia a architektúra systému	7
3.1	Celkový prehľad systému	7
3.2	Schéma spracovania	8
3.3	Technologický stack	8
4	Implementácia	10
4.1	Správa súborov a ukladanie stavov	10
4.2	Integrácia jazykového modelu a čistenie výstupu	10
4.3	Bezpečné spúšťanie Java kódu (Sandbox)	10
4.4	Dynamické programovanie pre optimalizáciu kontextu	11
5	Experimenty a Vyhodnotenie	12
5.1	Metodika A/B testovania	12
5.2	Výsledky experimentu	12
5.3	Analýza a diskusia	12
6	Záver	14

1 Úvod

V posledných rokoch sme svedkami masívneho rozvoja umelej inteligencie, pričom veľké jazykové modely (Large Language Models – LLM) začali radikálne transformovať mnohé odvetvia vrátane softvérového inžinierstva. Nástroje založené na LLM dokážu nielen asistovať pri písaní zdrojového kódu, ale aj navrhovať architektonické riešenia či refaktorovať existujúce programy. Kvalita a presnosť týchto výstupov však nie je garantovaná a kriticky závisí od kvality vstupného zadania – promptu.

Jednou z najefektívnejších metód, ako zvýšiť úspešnosť jazykových modelov pri riešení špecifických programovacích úloh, je obohatenie promptu o relevantný kontext z odbornej literatúry. Tento prístup však naráža na technický limit modelov, ktorým je obmedzená veľkosť kontextového okna. Keďže do promptu nie je možné vložiť celú knihu, vzniká optimalizačný problém: ako vybrať tie najužitočnejšie úryvky textu tak, aby sme maximalizovali šancu na vygenerovanie správneho kódu a zároveň neprekročili povolený počet tokenov.

Cieľom tohto ročníkového projektu je návrh a implementácia automatizovaného softvérového nástroja (pipeline-u), ktorý túto optimalizáciu vykonáva analyticky a deterministicky. Nástroj extrahuje znalosti z programátorskej literatúry, empiricky ohodnotí ich užitočnosť pomocou kompilácie a testovania vygenerovaného kódu v izolovanom prostredí a následne aplikuje algoritmus na riešenie problému batoha (Knapsack problem) pre zostavenie optimálneho "Final-Promptu".

Text práce je štruktúrovaný do piatich hlavných kapitol. **Druhá kapitola** analyzuje teoretické východiská, obmedzenia jazykových modelov a matematický základ optimalizácie. **Tretia kapitola** sa venuje celkovému návrhu a architektúre vytvoreného modulárneho systému. **Štvrtá kapitola** detailne popisuje konkrétnu softvérovú implementáciu, správu stavov a bezpečné spúšťanie vygenerovaného kódu. **Piata kapitola** prezentuje metodiku a výsledky záverečného A/B experimentu, v ktorom porovnávame úspešnosť modelu so základným zadáním a s optimalizovaným kontextom. Prácu uzatvára **Záver**, ktorý zhrňuje dosiahnuté výsledky a navrhuje možnosti ďalšieho rozvoja projektu.

2 Analýza problému a teoretické východiská

Táto kapitola sa zaoberá teoretickým pozadím využitia veľkých jazykových modelov (LLM) pri generovaní zdrojového kódu. Rozoberáme v nej limity súčasných modelov, metódy na hodnotenie vygenerovaného kódu a matematický prístup k optimalizácii kontextu prostredníctvom algoritmu diskkrétnej optimalizácie.

2.1 Veľké jazykové modely a Prompt Engineering

Veľké jazykové modely (Large Language Models, LLM) ako GPT-3.5, GPT-4 alebo modely rodiny Claude predstavujú prelom v oblasti umelej inteligencie a spracovania prirodzeného jazyka. Sú trénované na rozsiahlom korpuse textových dát vrátane zdrojových kódov vo viacerých programovacích jazykoch, vďaka čomu dokážu nielen rozumieť textu, ale aj generovať syntakticky a sémanticky korektný kód.

Kvalita výstupu, ktorý model vygeneruje, však kriticky závisí od vstupného zadania, tzv. promptu. *Prompt Engineering* je disciplína zameraná na návrh a optimalizáciu týchto vstupov. V kontexte programovania nestačí len popísať požadovanú funkcionality. Presnosť riešenia sa výrazne zvyšuje, ak model v prompte obdrží aj špecifické pravidlá, obmedzenia architektúry alebo ukážky správneho kódu.

2.2 Obmedzenia kontextového okna

Každý jazykový model má pevne stanovený limit na množstvo textu, ktoré dokáže spracovať v rámci jednej interakcie. Toto obmedzenie sa nazýva kontextové okno (context window) a merí sa v tokenoch (fragmentoch slov alebo znakov). Pre model GPT-3.5-turbo je základné kontextové okno obmedzené, hoci novšie modely (napríklad o200k_base pre GPT-4o) ponúkajú podstatne väčšiu kapacitu.

Pri snahe poskytnúť modelu ako kontext celú odbornú knihu (napr. *Effective Java*) rýchlo narazíme na tento limit. Prekročenie limitu vedie k chybe API, zatiaľ čo príliš dlhý kontext, ktorý sa zmestí do maxima, môže spôsobiť fenomén známy ako *Lost in the Middle*, kedy model ignoruje informácie umiestnené v strede rozsiahleho promptu. Z tohto dôvodu je nevyhnutné do promptu vyberať len tie najrelevantnejšie fragmenty textu.

2.3 Retrieval-Augmented Generation (RAG)

Prekonanie obmedzení kontextového okna a zamedzenie halucinácií modelu sa v modernej praxi rieši pomocou architektúry RAG (Retrieval-Augmented Generation). Tradičný prístup RAG funguje na báze vektorového vyhľadávania, kedy sa na základe otázky užívateľa nájdu najpodobnejšie dokumenty v databáze a tie sa pridajú do promptu.

V rámci nášho ročníkového projektu pristupujeme k tomuto konceptu inovatívne. Namiesto sémantického vyhľadávania vytvárame deterministický pipeline, ktorý apriórne (pred samotným testovaním) zmeria užitočnosť jednotlivých fragmentov literatúry a na základe týchto empirických metrík poskladá optimálny kontext.

2.4 Hodnotenie kvality vygenerovaného kódu

Aby sme vedeli kvantifikovať, aký užitočný bol daný fragment literatúry pre model, musíme objektívne zhodnotiť vygenerovaný kód. Využívame na to dataset programovacích úloh (v našom prípade benchmark *HumanEval* prispôbený pre jazyk Java).

Kvalitu hodnotíme na dvoch úrovniach:

- **Úspešnosť kompilácie (Compilation Rate):** Overuje, či model vygeneroval syntakticky správny kód, ktorý prejde kompilátorom (`javac`) bez chýb.
- **Úspešnosť testov (Test Pass Rate):** Prísnejšia metrika, ktorá spúšťa vygenerovaný kód voči vopred pripraveným testovacím prípadom (Unit testy) a kontroluje logickú správnosť riešenia.

Súčet úspešných testov pre daný fragment textu na vzorke úloh nám dáva empirickú hodnotu (váhu), ktorá vyjadruje "kvalitu" danej teoretickej rady.

2.5 Optimalizácia promptu: Diskrétny problém batoha

Po ohodnotení všetkých fragmentov literatúry stojíme pred optimalizačnou úlohou: ako z obrovského množstva textu vybrať takú podmnožinu, ktorá maximalizuje súhrnnú užitočnosť, ale jej celková veľkosť neprekročí limit kontextového okna (v tokenoch).

Tento problém je ekvivalentný klasickému *0/1 Knapsack Problem* (Problém batoha). Matematicky ho môžeme definovať nasledovne: Máme množinu n fragmentov literatúry, kde každý fragment má:

- **Hodnotu** v_i (úspešnosť testov, Test Pass Rate),
- **Váhu** w_i (veľkosť fragmentu v tokenoch).

Našou úlohou je nájsť binárny vektor $x = (x_1, x_2, \dots, x_n) \in \{0, 1\}^n$, ktorý určuje, či je i -ty fragment zahrnutý v prompte ($x_i = 1$) alebo nie ($x_i = 0$), tak, aby sme maximalizovali účelovú funkciu:

$$\max \sum_{i=1}^n v_i x_i$$

za podmienky kapacitného obmedzenia kontextového okna W :

$$\sum_{i=1}^n w_i x_i \leq W$$

Tento problém v projekte riešime pomocou dynamického programovania, čím zabezpečíme nájdenie globálneho optima. Asymptotická časová zložitosť tohto prístupu je $O(nW)$, čo je pre veľkosti modelov a množstvo textu postačujúce a garantuje nám optimálny kontext (Final-Prompt) pre následné testovanie.

3 Návrh riešenia a architektúra systému

Táto kapitola popisuje celkovú architektúru nášho riešenia. Systém bol navrhnutý ako modulárny dátový pipeline (reťazec na spracovanie dát) v programovacom jazyku Python. Každý krok reprezentuje samostatný spustiteľný skript, ktorý číta dáta zo štandardizovaného formátu JSON, vykoná potrebnú transformáciu alebo ohodnotenie, a výsledok opäť uloží. Tento prístup zaručuje jednoduchú rozšíriteľnosť, ladenie a stabilitu pri výskyte chýb.

3.1 Celkový prehľad systému

Architektúra je rozdelená do štyroch hlavných logických fáz, ktoré na seba striktne nadväzujú:

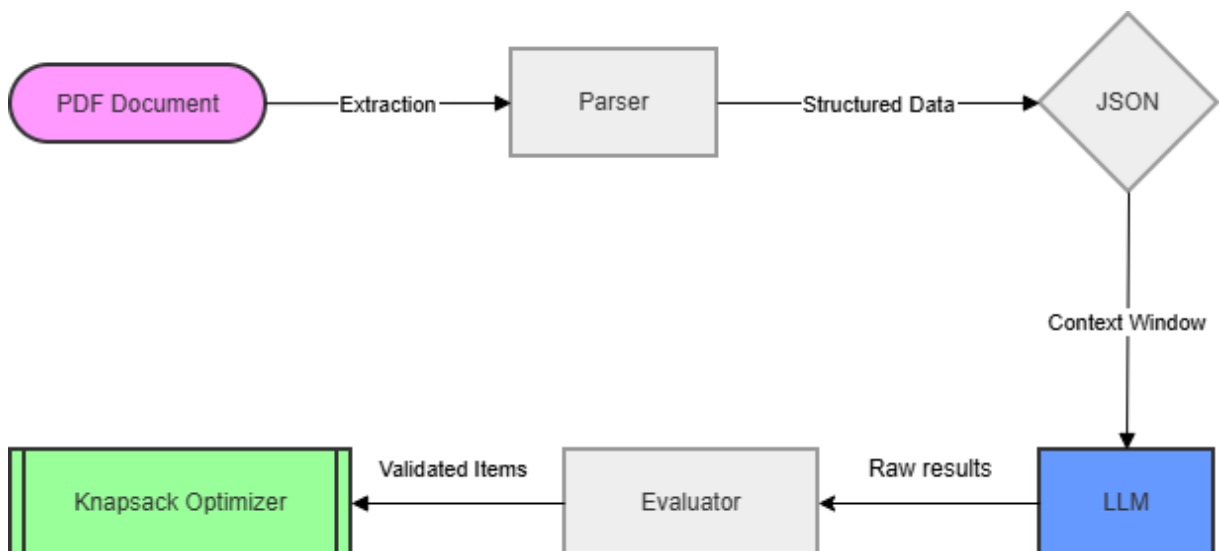
1. **Spracovanie literatúry a príprava dát (Parsing & Tokenization):** Vstupom je odborná kniha vo formáte PDF (napríklad *Effective Java*). Pomocou knižnice PyMuPDF sa z dokumentu extrahuje surový text, odstránia sa prebytočné biele znaky a text sa rozdelí na menšie logické celky (odstavce alebo vety). Každý fragment dostane unikátny identifikátor (hash) a je mu vypočítaná presná veľkosť v tokenoch (pomocou knižnice `tiktoken` a kódovania `cl100k_base`).
2. **Generovanie a evaluácia kódu (Predictor & Evaluator):** V tejto fáze systém prechádza pripravené fragmenty textu. Pre každý fragment a pre každú programovaciu úlohu z datasetu sa zostaví izolovaný prompt. Tento prompt je odoslaný cez API do modelu GPT-3.5. Následne prichádza kľúčová časť: vygenerovaný Java kód je izolovaný, očistený od Markdown formátovania a spustený v sandboxe (pomocou modulu `subprocess`).
 - Najprv prebehne pokus o kompiláciu pomocou nástroja `javac`.
 - V prípade úspešnej kompilácie sa kód spustí (`java`) a overí sa voči testom z datasetu.
3. **Optimalizácia kontextu (Knapsack Solver):** Keď sú všetky fragmenty ohodnotené metrikou úspešnosti, spustí sa algoritmus dynamického programovania. Ten zo stoviek fragmentov vyberie presne tú kombináciu, ktorá sa zmestí do vopred de-

finovaného limitu tokenov (napríklad 1200 tokenov) a zároveň maximalizuje súčet získaných bodov (tzv. Test Pass Rate).

4. **Finálny A/B Test (Final-Prompt Tester):** Vybrané fragmenty s najvyššou teoretickou hodnotou sú spojené do jedného veľkého kontextu, tzv. Final-Promptu. Systém následne vygeneruje záverečný report, v ktorom porovná úspešnosť LLM pri riešení úloh úplne bez kontextu (Baseline) a s využitím nášho optimalizovaného výberu z literatúry.

3.2 Schéma spracovania

Pre lepšiu ilustráciu toku dát a interakcie medzi modulmi uvádzame zjednodušenú schému.



Obr. 1: Architektúra modulárneho pipeline-u pre optimalizáciu promptu

3.3 Technologický stack

Na implementáciu vyššie popísaného riešenia sme zvolili nasledujúce technológie a nástroje:

- **Jazyk:** Python 3.10+ (pre riadiacu logiku a ML volania) a Java (pre kompiláciu a beh vygenerovaného kódu).
- **Rozhranie LLM:** Knižnica `openai` s architektonickou prípravou pre integráciu open-source modelov prostredníctvom abstrakčnej vrstvy `LiteLLM`.

- **Spracovanie dát:** Vlastná trieda `JSONHandler` postavená nad štandardnou knižnicou `json` pre perzistenciu stavov.

4 Implementácia

Táto kapitola sa zamerava na konkrétne detaily softvérovej implementácie navrhutej architektúry. Kód bol písaný s dôrazom na čitateľnosť, modularitu a princípy objektovo-orientovaného programovania v jazyku Python.

4.1 Správa súborov a ukladanie stavov

Pre zabezpečenie nezávislosti jednotlivých krokov pipeline-u sme sa rozhodli nevyužívať tradičnú relačnú databázu, ale stavový systém založený na formáte JSON. Trieda `JSONHandler` udržiava dáta v pamäti a automaticky aktualizuje vnútorné indexy podľa stavu fragmentu (napr. *parsed*, *tokenized*, *evaluated*). Vďaka tomu dokáže každý spúšťač skript (*runner*) rýchlo načítať len tie dáta, ktoré sú pripravené pre danú fázu spracovania. O konzistentné cesty k súborom a logickú separáciu adresárov sa stará trieda `DirectoryManager`, ktorá dynamicky vyhľadáva koreňový adresár projektu.

4.2 Integrácia jazykového modelu a čistenie výstupu

Na komunikáciu s jazykovým modelom bola vytvorená trieda `Predictor`. V základnom nastavení využíva knižnicu `openai` na volanie modelu z rodiny GPT-3.5. Systém je však štrukturálne pripravený aj na integráciu iných poskytovateľov (napríklad lokálnych open-source modelov) cez rozhranie `LiteLLM`, čo zabezpečuje flexibilitu do budúcnosti bez nutnosti prepisovať hlavnú logiku.

Keďže jazykové modely majú tendenciu generovať kód obalený v Markdown syntaxi (napríklad “`java`”), implementovali sme metódu `clean_llm_output`. Táto metóda deteguje a odstráni prebytočné formátovacie znaky a zabezpečí, že výstupom odovzdaným na evaluáciu je striktne čistý zdrojový kód.

4.3 Bezpečné spúšťanie Java kódu (Sandbox)

Najkritickejšou časťou evaluácie je kompilácia a spustenie kódu, ktorý vygenerovala umelá inteligencia. Keďže model môže vygenerovať kód obsahujúci nekonečné cykly alebo vážne syntaktické chyby, museli sme zaistiť bezpečné prostredie pre jeho beh.

Trieda `Evaluator` tento problém rieši vytvorením dočasného adresára `temp_java`, do

ktorého zapíše vygenerovaný kód spolu s testovacou obálkou. Následne pomocou štandardnej Python knižnice `subprocess` inicializuje nové procesy operačného systému:

- Spustí kompilátor `javac` s časovým limitom 15 sekúnd.
- V prípade úspechu spustí `java Main` s časovým limitom 10 sekúnd.

Ak proces presiahne stanovený čas (*Timeout*) alebo skončí chybou (nenulový návratový kód kompilácie), `Evaluator` bezpečne proces ukončí a fragmentu priradí nulové skóre pre daný test.

4.4 Dynamické programovanie pre optimalizáciu kontextu

Srdcom výberu ideálneho kontextu je trieda `KnapsackSolver`. Keďže hľadáme optimálne riešenie diskretného problému 0/1 batoha, využívame prístup dynamického programovania. Tabuľka stavov (`dp`) sa iteratívne naplňa podľa pravidla, či sa aktuálny fragment (s váhou v tokenoch) zmestí do dočasného limitu kontextového okna a či jeho pridanie zvýši celkovú metriku úspešnosti.

Ukážka jadra algoritmu v jazyku Python:

Listing 1: Implementácia algoritmu Knapsack pomocou dynamického programovania

```
1 dp = [[0.0] * (W + 1) for _ in range(n + 1)]
2
3 for i in range(1, n + 1):
4     item = items[i-1]
5     weight = item["metadata"]["tokens"]["cl100k_base"]
6     value = item["metadata"]["metrics"][self.metric_name] or 0.0
7
8     for w in range(W + 1):
9         if weight <= w:
10             dp[i][w] = max(dp[i-1][w], dp[i-1][w - weight] + value)
11         else:
12             dp[i][w] = dp[i-1][w]
```

Po naplnení matice systém prejde tabuľku spätne od konca a identifikuje presné identifikátory (ID) fragmentov textu, ktoré tvoria optimálny výsledok. Tento prístup je plne deterministický a na rozdiel od heuristických metód garantuje nájdenie globálneho optima vzhľadom na zvolený cieľ (napr. maximalizácia `test_pass_rate`).

5 Experimenty a Vyhodnotenie

Táto kapitola prezentuje výsledky testovania navrhnutého systému. Cieľom experimentu bolo overiť hlavnú hypotézu projektu: či pripojenie algoritmicke vybraného kontextu z odbornej literatúry (Final-Prompt) zlepši schopnosť jazykového modelu riešiť programovacie úlohy v porovnaní so základným prístupom bez kontextu (Baseline).

5.1 Metodika A/B testovania

Pre objektívne vyhodnotenie sme implementovali triedu `FinalPromptTester`, ktorá vykonáva automatizované A/B testovanie. Experiment prebiehal za nasledovných podmienok:

- **Model:** `gpt-3.5-turbo` (s teplotou nastavenou na 0.2 pre deterministickejšie výstupy).
- **Dataset:** Náhodná vzorka 5 úloh z benchmarku `HumanEval` prispôbeného pre jazyk Java.
- **Zdrojový kontext:** Optimalizovaný výber fragmentov z knihy *Effective Java* pomocou Knapsack algoritmu (s limitom 1200 tokenov).

Každá úloha bola modelom riešená dvakrát. V prvom prípade dostal model len zadanie úlohy (Baseline). V druhom prípade dostal rovnaké zadanie rozšírené o Final-Prompt. Oba vygenerované zdrojové kódy boli následne kompilované a testované voči rovnakej sade Unit testov.

5.2 Výsledky experimentu

Súhrnné výsledky experimentu sú zaznamenané v Tabuľke 1. Obe metriky (úspešnosť kompilácie aj úspešnosť prechodu testami) sú uvedené ako pomer úspešných pokusov k celkovému počtu úloh.

5.3 Analýza a diskusia

Ako môžeme vidieť z výsledkov, experiment na testovacej vzorke skončil remízou (zhodný výsledok pre kompiláciu aj logické testy). Tento neutrálny výsledok je z hľadiska výskumu

Prístup	Úspešná kompilácia	Úspešné testy (Pass Rate)
Baseline (Bez kontextu)	4 / 5 (80 %)	3 / 5 (60 %)
Final-Prompt (Knapsack kontext)	4 / 5 (80 %)	3 / 5 (60 %)

Tabuľka 1: Porovnanie úspešnosti generovania kódu (Baseline vs. Final-Prompt)

mimoriadne zaujímavý a poukazuje na dôležitý fenomén pri práci s RAG (Retrieval-Augmented Generation) systémami v programovaní.

Zhodný výsledok je primárne spôsobený nesúlalom medzi charakterom použitých dát:

1. **Algoritmický charakter datasetu:** Benchmark HumanEval je zameraný na základné algoritmické úlohy (napr. hľadanie prvočísel, reverzácia reťazca, práca s poľami). Pre takýto typ úloh má model GPT-3.5 už v základnej konfigurácii dostatok vedomostí a prejavuje sa tzv. *Ceiling Effect* (efekt stropu), kedy pridaný kontext nedokáže výraznejšie vylepšiť už takmer optimálny výkon.
2. **Architektonický charakter literatúry:** Kniha *Effective Java* obsahuje komplexné rady týkajúce sa softvérovej architektúry, návrhových vzorov, bezpečnosti vlákien a generického programovania. Tieto hlboké znalosti sú pre jednoduché algoritmické úlohy nadbytočné a model ich v tomto kontexte nedokáže využiť.

Aj napriek tomu, že Final-Prompt priamo nezvýšil skóre na tomto špecifickom datasete, experiment úspešne preukázal plnú funkčnosť celého pipeline-u. Systém dokázal vybrať kontext, spojiť ho s promptom, získať odpoveď a bez chýb ju vyhodnotiť. Pre preukázanie prínosu literatúry typu *Effective Java* bude v budúcnosti potrebné zvoliť dataset zameraný na refactoring alebo návrh tried, kde sa znalosti z knihy naplno prejavujú.

6 Záver

Cieľom tohto ročníkového projektu bolo navrhnuť a implementovať automatizovaný systém na optimalizáciu promptov pre veľké jazykové modely v doméne programovania, s využitím externej odbornej literatúry.

Tento cieľ sa podarilo úspešne naplniť vytvorením modulárneho softvérového riešenia v jazyku Python. Zrealizovaný pipeline dokáže samostatne spracovať textové dokumenty (PDF), rozdeliť ich na logické fragmenty a ohodnotiť ich užitočnosť na základe úspešnosti vygenerovaného Java kódu v prísnom, izolovanom prostredí (sandbox). Následne systém využíva pokročilý algoritmus dynamického programovania (0/1 Knapsack Problem) na výber tých najlepších úryvkov tak, aby sa neprekročil limit kontextového okna jazykového modelu.

Hoci finálny experiment na malej vzorke algoritmických úloh ukázal zhodnú výkonnosť medzi základným modelom a modelom s optimalizovaným kontextom, analýza tohto javu priniesla cenné zistenia o vplyve charakteru datasetu na úspešnosť RAG systémov.

Vyvinutý systém je vysoko prispôsobiteľný a pripravený na ďalšie nasadenie. Vďaka integrácii vrstvy LiteLLM je možné do projektu jednoducho zapojiť rôzne (aj lokálne) jazykové modely. Práca tvorí pevný základ pre ďalší výskum, najmä v oblasti testovania komplexnejších programovacích úloh, kde by sa hlboké znalosti z odbornej literatúry mohli plne zhodnotiť.